# VISVESVARAYA TECHNOLOGICAL UNIVERSITY

**BELAGAVI-590018, KARNATAKA**

## ENTREPRENEURSHIP PROJECT REPORT

ON

## "Aircraft avionics System architecture simulation"

**Submitted by**

Name: Nidhi Bhadoria, USN: 1CR22EC151
Name: Adam Nabeel, USN:1CR22EC007

**Under the guidance of**

Name: Dr Richa Tengshe
Assistant Professor
**Department of Electronics and Communication Engineering**

August-December 2024


**Department Of Electronics and Communication Engineering**

**CMR INSTITUTE OF TECHNOLOGY**

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI,

BENGALURU-560037

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING**

# CERTIFICATE

This is to certify the Entrepreneurship Project Report entitled “Aircraft avionics System architecture simulation", prepared by **Nidhi Bhadoria**, bearing **USN 1CR22EC151** and **Adam Nabeel**, bearing **USN 1CR22EC007**, bona fide students of **CMR Institute of Technology, Bengaluru** in partial fulfillment of the requirements for the award of **Bachelor of Engineering in Electronics and Communication Engineering** of the **Visvesvaraya Technological University, Belagavi-590018** during the academic year 2024-25.

This is certified that all the corrections and suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The Entrepreneurship Project has been approved as it satisfies the academic requirements prescribed for the said degree.

| | |
|---|---|
| **----------** | **----------** |
| **Signature of Guide** | **Signature of HOD** |
| **Dr. Richa Tengshe** | **Dr. Pappa M** |
| **Assistant Professor** | **Professor & HoD** |
| **Dept. of ECE, CMRIT** | **Dept. of ECE, CMRIT** |

Date: 17 Dec 2024

# TO WHOMSOEVER IT MAY CONCERN

This is to acknowledge the successful completion of project by the following students of CMRIT, College, ITPL Road. Bangalore for the development of “Aircraft avionics System architecture simulation".

**1. Nidhi Bhadoria :1CR22EC151**
**2. Adam Nabeel :1CR22EC007**

The students have worked on the elements for the development of “Aircraft avionics System architecture simulation”. This software will be used for development activities in manned and unmanned aircraft platforms.

**Mentor -Mr. Prashantsingh Bhadoria**
**General Manager- Program Management-TASL -AP&S**

For Tata Advanced Systems Limited

**Navisha Hegde**
**Manager - Human Resources**

## ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without mentioning the people whose proper guidance and encouragement has served as a beacon and crowned my efforts with success. I take an opportunity to thank all the distinguished personalities for their enormous and precious support and encouragement throughout the duration of this seminar.

I take this opportunity to express my sincere gratitude and respect to **CMR Institute of Technology, Bengaluru** for providing me an opportunity to present my mini project.

I have a great pleasure in expressing my deep sense of gratitude to **Dr. Sanjay Jain,** Principal, CMRIT, Bangalore, for his constant encouragement.

I would like to thank **Dr. Pappa M,** HoD, Department of Electronics and Communication Engineering, CMRIT, Bangalore, who shared her opinion and experience through which I received the required information crucial for the entreprenurship project.

I consider it a privilege and honor to express my sincere gratitude to my guide **Dr. Richa Tengshe, Assistant Professor**, Department of Electronics and Communication Engineering, for the valuable guidance throughout the tenure of this review.

I also extend my thanks to the faculties of Electronics and Communication Engineering Department who directly or indirectly encouraged me.

Finally, I would like to thank my parents and friends for all their moral support they have given me during the completion of this work.

**Nidhi Bhadoria (1CR22EC151)**

**Adam Nabeel (1CR22EC007)**

# ABSTRACT

The evolution of modern aviation demands sophisticated tools for simulating and evaluating avionics systems to ensure performance, reliability, and safety. This project presents an **Aircraft Avionics System Architecture Simulation** developed using MATLAB and Simulink. The simulation provides a comprehensive platform for analysing and testing various avionics subsystems critical to aircraft operation.

Key subsystems incorporated into the simulation include:

1. **Flight Path Control**: Simulates the aircraft's trajectory to evaluate path optimization and ensure stable navigation under different flight scenarios.
2. **Altitude and Navigation Control**: Models the systems responsible for maintaining precise altitude and accurate navigation, crucial for operational efficiency and safety.
3. **Pitch and Pitch Error Control**: Focuses on pitch dynamics, providing mechanisms for error correction to maintain desired pitch angles and enhance flight stability.

The project leverages MATLAB's computational capabilities and Simulink's graphical simulation environment to develop a modular and scalable avionics architecture. The system is designed to accommodate real-world data integration and allow for the addition of advanced features, such as sensor feedback and adaptive control algorithms.

The project is designed with a modular approach, allowing individual subsystems to be independently analyzed and tested before integration into the complete avionics architecture. This modularity supports iterative development, making it easier to refine specific components and assess their interactions with the overall system. Additionally, the simulation incorporates realistic environmental conditions and operational constraints to provide a more accurate representation of real-world scenarios.

This simulation system serves as an invaluable tool for understanding and validating avionics system architectures, with potential applications in educational training, prototyping, and the development of next-generation avionics technologies.

**CONTENTS**

# Ch 1.1: INTRODUCTION

Avionics, short for "aviation electronics," refers to the electronic systems used in aircraft, satellites, and spacecraft. These systems play a critical role in ensuring the safe and efficient operation of modern aircraft, encompassing navigation, communication, monitoring, and control systems. The complexity and interconnectivity of avionics systems necessitate robust simulation environments to design, analyse, and validate their performance.

This project focuses on the **Aircraft Avionics System Architecture Simulation**, leveraging MATLAB and Simulink to create a modular and scalable platform for studying and optimizing avionics subsystems. The simulation replicates key avionics functionalities, including flight path control, altitude and navigation control, and pitch and pitch error control.

Figure 1.1 : Avionics in Aircraft

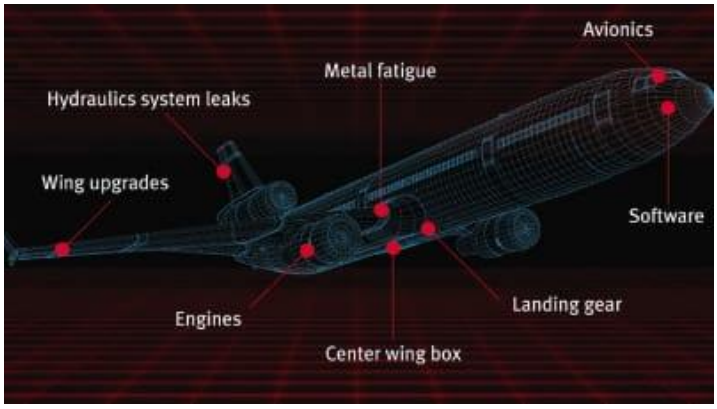


Figure 1.2 : Aircraft body architecture

## 1.2: KEY AVIONICS SYSTEMS INCORPORATED

1. Flight path control:

   Flight path control ensures the aircraft follows a predefined trajectory or adjusts dynamically based on input parameters. This involves solving differential equations of motion and incorporating aerodynamic forces and environmental conditions.

   $$\vec{F}=m.\vec{a} \qquad \text{(equation 1.2.1)}$$

   where $\vec{F}$ represents the net force acting on the aircraft, mmm is the mass of the aircraft, and $\vec{a}$ is the acceleration. Trajectory optimization often uses kinematic equations and constraints derived from navigation goals and operational limits.

2. Altitude and Navigation Control:

   Altitude control maintains the desired flight level by managing thrust, lift, and drag, while navigation ensures the aircraft adheres to its intended route using guidance systems.

   $$L=W \qquad \text{(equation 1.2.2)}$$

   Where L is the lift force and W is the weight of the aircraft. The lift force depends on factors such as air density (ρ), wing area (A), and velocity (v):

   $$L=(1/2)\cdot\rho\cdot v^2\cdot A\cdot C_L \qquad \text{(equation 1.2.3)}$$

   Here, $C_L$ is the lift coefficient.

   Navigation employs algorithms such as Proportional-Integral-Derivative (PID) controllers to correct course deviations.

3. Pitch and Pitch Error Control:

   Pitch control manages the nose angle of the aircraft relative to the horizontal axis, critical for maintaining stability during ascent, descent, and level flight. Pitch error correction is implemented to minimize deviations from the desired pitch angle.

   $$I\cdot\alpha=\tau_{input}-\tau_{aero} \qquad \text{(equation1.2.4)}$$

   where I is the moment of inertia about the pitch axis, α is the angular acceleration, $\tau_{input}$ is the control torque, and $\tau_{aero}$ represents aerodynamic torques. Error correction uses feedback control systems, such as PID controllers, to adjust control surfaces like the elevator.

# Ch 2.1 : IMPORTANCE OF SIMULATION

Simulating avionics systems is a cornerstone of modern aerospace engineering, and its significance is particularly evident in the context of this **Aircraft Avionics System Architecture Simulation** project. By leveraging MATLAB and Simulink, the project creates a virtual environment to model, analyse, and optimize critical avionics subsystems, such as flight path control, altitude and navigation control, and pitch error correction. This approach provides a cost-effective and risk-free method for testing complex scenarios and system behaviours that would be difficult or unsafe to replicate in real-world conditions.

The simulation framework allows for the evaluation of various subsystem interactions, ensuring seamless integration within the overall avionics architecture. For instance, flight path adjustments can be analysed in conjunction with pitch dynamics to understand how these systems affect one another. By incorporating real-world constraints, such as environmental disturbances and system failures, the simulation provides insights into the performance and reliability of these subsystems under diverse operating conditions.

Moreover, the modular design of the simulation enables rapid iteration and optimization of individual components. Engineers can experiment with different configurations, refine control algorithms, and validate their performance before integrating them into the larger system. This iterative process not only accelerates development but also reduces the need for expensive physical prototypes and flight tests.

The project also highlights the potential for expanding the simulation environment with advanced features, such as hardware-in-the-loop (HIL) testing, real-time sensor data integration, and machine learning-based control systems. These extensions would allow the simulation to evolve into a more comprehensive tool for prototyping and system verification.

## Ch 2.2 :SOME BASIC DEFINTIONS RELATED TO FLIGHT CONTROL SYSTEMS

- **Flight Control System (FCS)**: A system that manages the aircraft's flight dynamics by adjusting control surfaces and power to achieve desired flight conditions. It includes manual, automatic, and feedback controls for maintaining stability, altitude, and trajectory.

- **Pitch**: The rotation of an aircraft around its lateral axis, which controls the aircraft's nose-up or nose-down attitude. It is typically controlled by the **elevator** on the tail plane.

- **Yaw**: The rotation of an aircraft around its vertical axis, which controls the aircraft's left or right movement (turning). It is controlled by the **rudder** on the vertical stabilizer.

- **Roll**: The rotation of an aircraft around its longitudinal axis, controlling the tilt of the wings. It is controlled by the **aileron** on the wings.

- **Altitude**: The vertical distance of the aircraft above the Earth's surface, usually measured in feet or meters. It is an important parameter for determining flight levels and ensuring safe separation from the ground and other aircraft.

- **PID Controller**: A feedback control loop widely used in flight control systems that adjusts the control surface positions based on the proportional, integral, and derivative components of the error between desired and actual values (e.g., altitude, speed, or flight path).

- **Throttle**: The control that regulates the engine power of the aircraft, impacting speed and altitude. It is adjusted to maintain a desired performance level during flight.

- **Transfer Function**: A mathematical representation of the relationship between the input and output of a system. In flight control systems, transfer functions model the dynamic behaviour of the aircraft's components, such as altitude or navigation control.

- **Navigation**: The process of determining and controlling the aircraft's position, speed, and course relative to a desired flight path. This involves systems like GPS, inertial navigation, and waypoints.

- **Flight Path**: The trajectory that an aircraft follows during its flight, which includes its altitude, speed, and direction. Flight path control ensures the aircraft stays on its intended route.

- **Feedback Loop**: A system that automatically adjusts its operations based on the difference between the desired and actual output. In flight control systems, feedback loops ensure that the aircraft maintains its stability and follows the desired flight path.

- **Autopilot**: A system that controls the aircraft's flight path with little or no input from the pilot. Autopilot systems typically include altitude hold, navigation, and heading control features.

- **Control Surface**: A movable part of the aircraft that is used to control the aircraft's orientation, such as the ailerons, elevators, and rudders.

- **Error Signal**: The difference between the desired value (e.g., target altitude) and the actual value (e.g., current altitude). The error signal is used in control systems like PID to adjust the system's output.

- **Flight Envelope**: The range of operating conditions (such as speed, altitude, and load factor) within which the aircraft is designed to safely operate. The flight control system helps ensure the aircraft stays within this envelope.

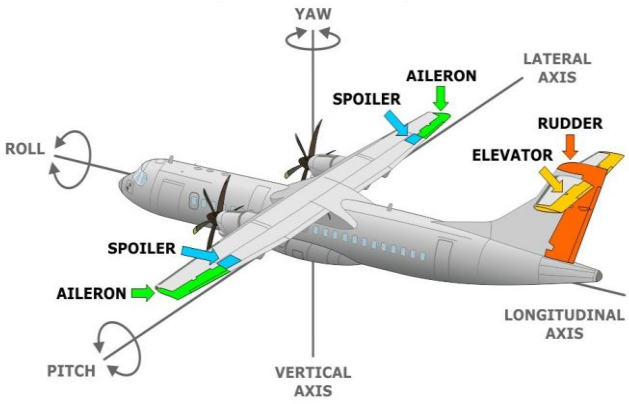


Figure 2.2.1 :Basic Flight controls

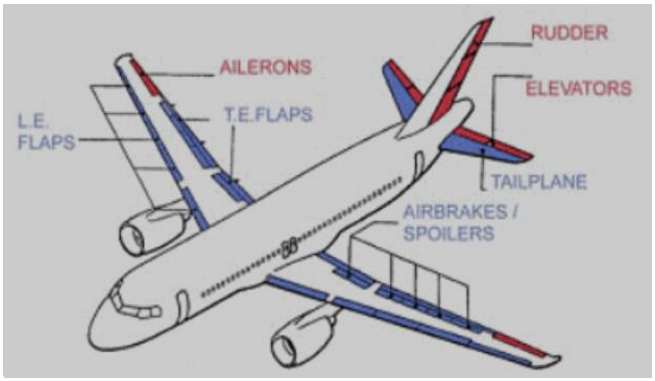


Figure 2.2.2 : Basic Flight architecture

# CH 3.1: PITCH AND PITCH CONTROL IN AVIONICS

Pitch refers to the rotation of an aircraft around its lateral axis, which runs from wingtip to wingtip. This motion causes the nose of the aircraft to move up or down and is one of the primary control movements in aircraft flight. In avionics, pitch control is essential for maintaining stability during different phases of flight, such as take-off, cruising, and landing. The pitch angle directly influences the aircraft's attitude relative to the horizon and affects its flight path, altitude, and overall performance.

**Pitch Control Systems** in avionics typically involve the use of control surfaces, such as the elevator, which adjusts the aircraft's angle of attack and, consequently, its pitch. The control system continuously receives inputs from sensors, like attitude indicators, and uses feedback loops to adjust the control surfaces to maintain the desired pitch angle. This can be achieved through manual control by the pilot or through automated systems like autopilots.

**Pitch Error Control** is equally important in ensuring the aircraft maintains the desired pitch angle despite disturbances like turbulence or changes in aircraft load. A pitch error occurs when the actual pitch deviates from the desired value, potentially affecting flight stability and safety. Avionics systems use **feedback control mechanisms**, such as **PID controllers** (Proportional-Integral-Derivative), to minimize these errors. These systems compare the desired pitch angle to the actual one, correcting any discrepancies by adjusting the control surfaces.

Pitch and pitch error control are fundamental to aircraft stability, and avionics systems are designed to ensure precise and reliable management of the aircraft's pitch to maintain optimal flight performance.

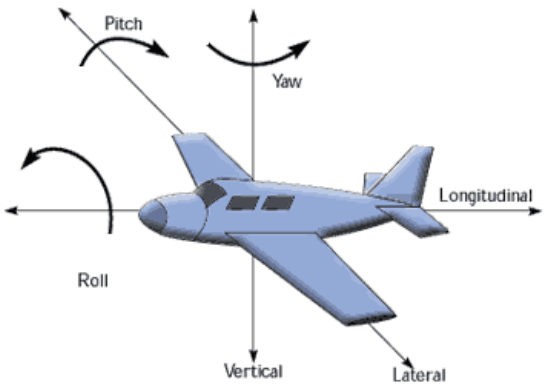


Figure 3.1

# Ch 3.2: ALTITUDE AND NAVIGATION

In avionics, **altitude** and **navigation** are two critical aspects of flight control, ensuring that an aircraft maintains its desired altitude and follows the intended flight path accurately and safely.

**Altitude Control** is the system responsible for maintaining the aircraft at a specific height above the Earth's surface. It is crucial for ensuring safe separation between aircraft, especially in crowded airspace. The avionics systems that manage altitude use a variety of sensors, such as **altimeters** (which measure atmospheric pressure) and **radar altimeters** (which measure height above the ground). The flight management system (FMS) adjusts the aircraft's **thrust** and **control surfaces** to maintain the desired altitude. In automated systems like autopilots, the FMS continuously compares the current altitude to the target altitude and makes adjustments as needed.

**Navigation** refers to the process of determining the aircraft's position and directing it along a predefined route. Modern avionics systems employ a combination of **Global Navigation Satellite Systems (GNSS)**, **Inertial Navigation Systems (INS)**, and **radio navigation aids** to provide continuous and accurate position information. The aircraft's **Flight Management System (FMS)** uses this data to calculate the optimal route, ensuring the aircraft stays on course. Navigation systems are often integrated with autopilot to automatically adjust the aircraft's heading, speed, and altitude to follow the planned trajectory. Together, **altitude and navigation control** ensure that the aircraft operates efficiently and safely, maintaining a precise flight path while avoiding altitude deviations. These systems are integral to modern avionics, enabling both manual and automated control of the aircraft's trajectory and position during flight.

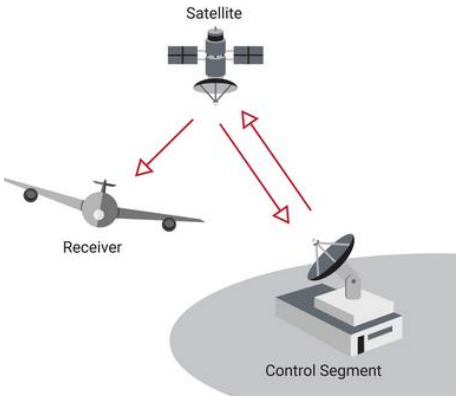


Figure 3.2.1 : GNSS Constellation

# Ch 3.3: FLIGHT PATH

In avionics, **flight path** refers to the trajectory that an aircraft follows as it moves through the air, which is determined by its position, speed, and altitude. Maintaining a precise flight path is crucial for ensuring safe, efficient, and accurate navigation, especially when flying through complex airspace or when performing specific manoeuvres like take-off, cruising, and landing.

The **flight path control system** in avionics uses data from various sensors, such as **Global Navigation Satellite Systems (GNSS)**, **Inertial Navigation Systems (INS)**, and **Flight Management Systems (FMS)**, to determine the optimal route for the aircraft. The system continuously monitors the aircraft's position and adjusts its heading, altitude, and speed to stay on the intended path.

The flight path can be adjusted manually by the pilot or automatically by the autopilot system. Automated systems use advanced algorithms, including **trajectory prediction and path optimization**, to adjust the aircraft's course in real-time. These systems take into account factors like wind speed, air traffic, and weather conditions to modify the flight path, ensuring the most efficient and safe route is followed.

In summary, the **flight path** is a dynamic, real-time representation of an aircraft's journey, managed by avionics systems to ensure safe and efficient flight. The integration of **navigation, altitude control**, and **trajectory optimization** is essential in maintaining the desired flight path throughout the aircraft's journey.

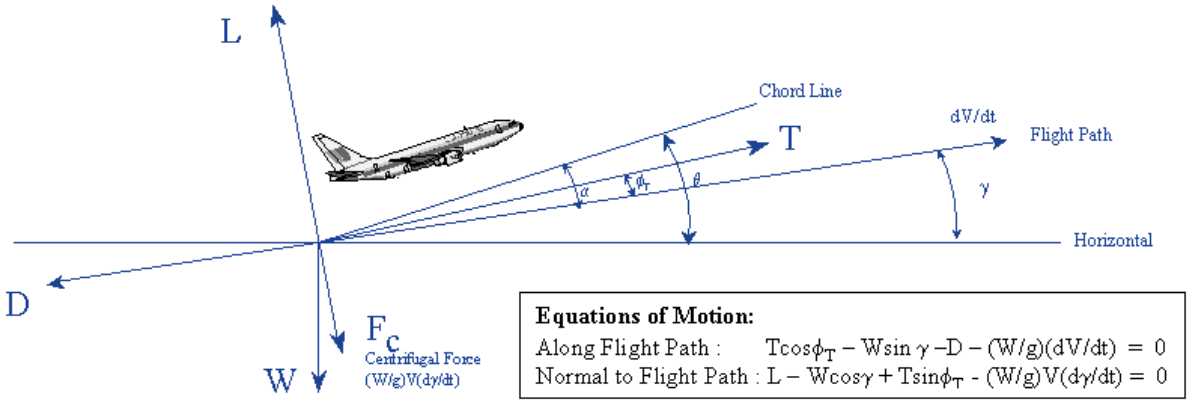


Figure 3.3.1 : Approximate Flight path model

# CH 4 : MATLAB SIMULATIONS

## 4.1 : Pitch control dynamics

**CODE:**

```
clc
clear all
close all
% Define the model name

modelName = 'Flight_Avionics_Model';
% Check if the model already exists and close it if necessary
if bdIsLoaded(modelName)
close_system(modelName, 0); % Close the model without saving
delete_system(modelName); % Delete the model from memory
end
% Create a new model
new_system(modelName); % Create new model
open_system(modelName); % Open the model
% Step 1: Aircraft Dynamics (using a Transfer Function for pitch control)
add_block('simulink/Continuous/Transfer Fcn', [modelName, '/Pitch Dynamics']);
set_param([modelName, '/Pitch Dynamics'], 'Numerator', '[1]', 'Denominator', '[1 1
0]');
% Step 2: Add Step Input for Elevator Deflection (Input signal)
add_block('simulink/Sources/Step', [modelName, '/Elevator Deflection']);
set_param([modelName, '/Elevator Deflection'], 'Time', '1', 'Before', '0', 'After',
'10');
% Step 3: Create Sum block to calculate error (Desired - Actual)
add_block('simulink/Math Operations/Sum', [modelName, '/Error Calculation']);
set_param([modelName, '/Error Calculation'], 'Inputs', '|+-'); % Subtract actual from
desired
% Step 4: PID Controller to control the aircraft's pitch
add_block('simulink/Continuous/PID Controller', [modelName, '/PID Controller']);
set_param([modelName, '/PID Controller'], 'P', '1', 'I', '0', 'D', '0');
% Step 5: Add Scope to visualize results (Pitch angle and error)
add_block('simulink/Sinks/Scope', [modelName, '/Pitch Angle Scope']);
add_block('simulink/Sinks/Scope', [modelName, '/Error Scope']);
% Step 6: Connect Blocks (Signal Flow)
% Connect Step input (Elevator Deflection) to Error Calculation block
add_line(modelName, 'Elevator Deflection/1', 'Error Calculation/1'); % Desired input
% Connect Pitch Dynamics (Actual output) to Error Calculation (Negative input)
add_line(modelName, 'Pitch Dynamics/1', 'Error Calculation/2'); % Actual output
% Connect Error Calculation to PID Controller (Error signal to PID)
add_line(modelName, 'Error Calculation/1', 'PID Controller/1');
% Connect PID Controller output to Pitch Dynamics input (Elevator command)
add_line(modelName, 'PID Controller/1', 'Pitch Dynamics/1');
% Connect Pitch Dynamics output to Scope (Pitch Angle)
add_line(modelName, 'Pitch Dynamics/1', 'Pitch Angle Scope/1');
% Connect Error Calculation output to Error Scope
add_line(modelName, 'Error Calculation/1', 'Error Scope/1');
% Step 7: Set Simulation Parameters
set_param(modelName, 'Solver', 'ode45', 'StartTime', '0', 'StopTime', '50');
```

```
% Step 8: Run the Simulation
sim(modelName);
% Display the results (Scope)
open_system([modelName, '/Pitch Angle Scope']);
open_system([modelName, '/Error Scope']);
```

**OUTPUT :**

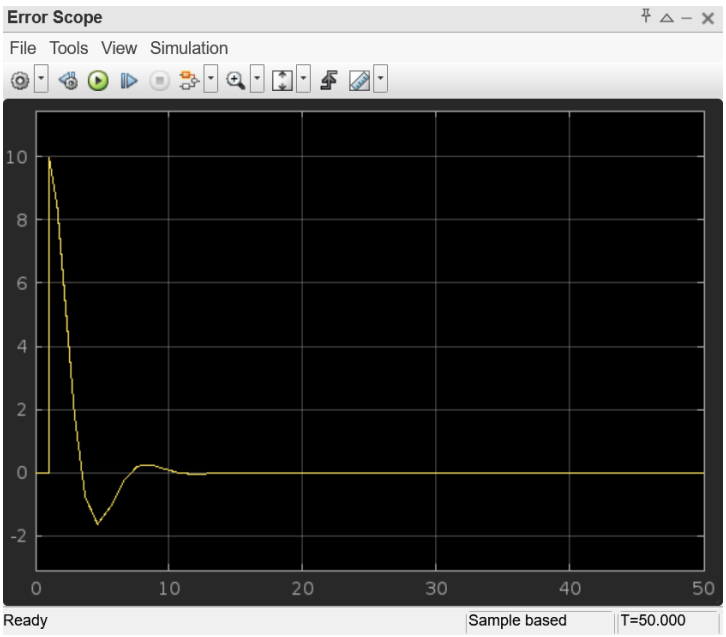


Figure 4.1.1 : Error Scope

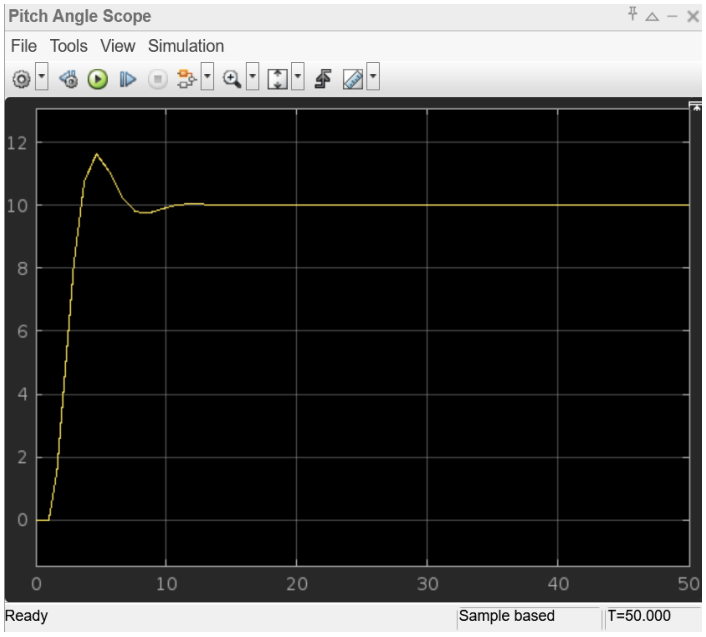


Figure 4.1.2: Pitch Angle Scope

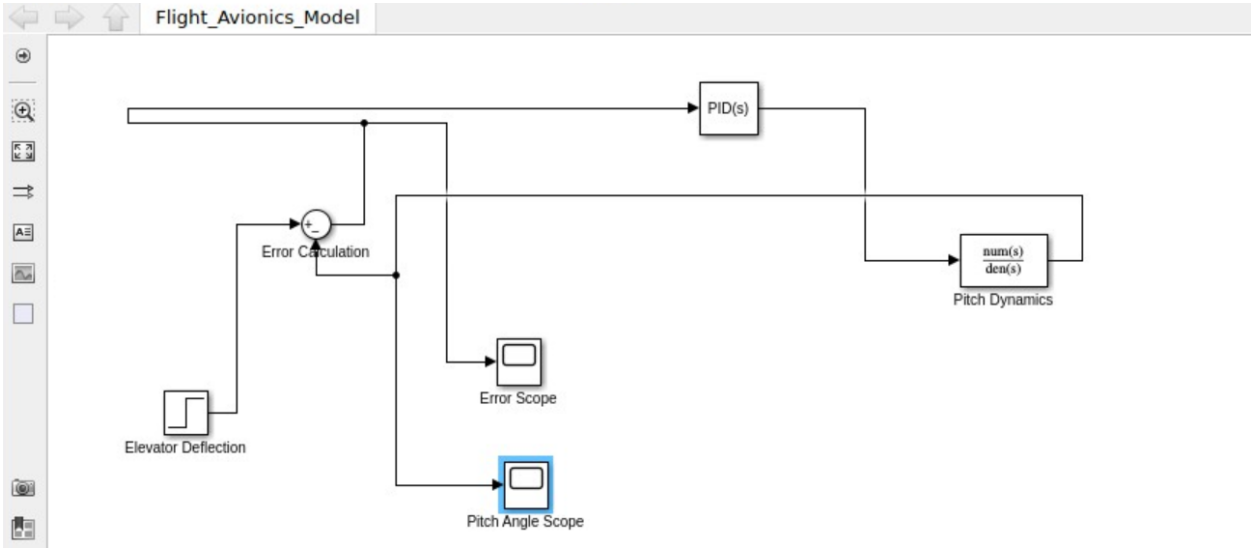


Figure 3.1.3 : Simulation of Pitch Control

**EXPLANATION OF CODE :**

This MATLAB code simulates a basic **flight avionics architecture** using Simulink, focusing on **pitch control dynamics** of an aircraft. It begins by initializing the environment, clearing previous variables and models, and defining a new Simulink model named **Flight_Avionics_Model**. The model is constructed step-by-step, starting with the addition of a **Transfer Function** block to represent the aircraft's pitch dynamics. A **step input** is used to simulate elevator deflection, which influences the aircraft's pitch. The code then includes a **Sum block** to calculate the error between the desired and actual pitch angles, which is used by a **PID controller** to adjust the elevator deflection and maintain the desired pitch. To visualize the simulation, two **Scope blocks** are added to monitor the pitch angle and error over time. These blocks are interconnected, creating a signal flow where the output of the error calculation is sent to the PID controller, which adjusts the pitch dynamics. Finally, the simulation parameters are set, including the solver and time range, and the simulation is executed. The results are displayed through the Scope blocks, showing how the system responds to the elevator deflection input in terms of pitch control and error correction. This approach helps analyze the effectiveness of the pitch control system in maintaining aircraft stability.

## Ch 4.2 : Altitude and Navigation

**CODE :**

```
% MATLAB Code to Simulate Altitude Regulation and Navigation of an Aircraft in
Simulink Online
clc
clear all
close all
% Define the model name
modelName = 'Aircraft_Altitude_Navigation_Model';
% Check if the model already exists and close it if necessary
if bdIsLoaded(modelName)
close_system(modelName, 0); % Close the model without saving
delete_system(modelName); % Delete the model from memory
end
% Create a new model
new_system(modelName); % Create new model
open_system(modelName); % Open the model
% Step 1: Aircraft Altitude Dynamics (using a Transfer Function)
add_block('simulink/Continuous/Transfer Fcn', [modelName, '/Altitude Dynamics']);
set_param([modelName, '/Altitude Dynamics'], 'Numerator', '[1]', 'Denominator', '[10
1]');
% Step 2: Add Step Input for Desired Altitude (Altitude Reference)
add_block('simulink/Sources/Step', [modelName, '/Desired Altitude']);
set_param([modelName, '/Desired Altitude'], 'Time', '1', 'Before', '1000', 'After',
'2000'); % Desired altitude
% Step 3: Create Sum block to calculate altitude error (Desired - Actual)
add_block('simulink/Math Operations/Sum', [modelName, '/Altitude Error
Calculation']);
set_param([modelName, '/Altitude Error Calculation'], 'Inputs', '|+-'); % Subtract
actual from desired
% Step 4: PID Controller for Altitude Control
add_block('simulink/Continuous/PID Controller', [modelName, '/Altitude PID
Controller']);
set_param([modelName, '/Altitude PID Controller'], 'P', '1', 'I', '0.1', 'D', '0.1');
% Step 5: Add Scope to visualize altitude and error
add_block('simulink/Sinks/Scope', [modelName, '/Altitude Scope']);
add_block('simulink/Sinks/Scope', [modelName, '/Altitude Error Scope']);
% Step 6: Connect Blocks for Altitude Regulation
add_line(modelName, 'Desired Altitude/1', 'Altitude Error Calculation/1'); % Desired
altitude
add_line(modelName, 'Altitude Dynamics/1', 'Altitude Error Calculation/2'); % Actual
altitude
add_line(modelName, 'Altitude Error Calculation/1', 'Altitude PID Controller/1'); %
Error to PID
add_line(modelName, 'Altitude PID Controller/1', 'Altitude Dynamics/1'); % PID to
altitude dynamics
add_line(modelName, 'Altitude Dynamics/1', 'Altitude Scope/1'); % Altitude output to
scope
```

```
add_line(modelName, 'Altitude Error Calculation/1', 'Altitude Error Scope/1'); %
Error to scope
% Step 7: Navigation Control (Waypoint navigation)
% Create a simple Proportional Control for horizontal navigation
add_block('simulink/Math Operations/Sum', [modelName, '/Navigation Error
Calculation']);
set_param([modelName, '/Navigation Error Calculation'], 'Inputs', '|+-'); %
Navigation error
add_block('simulink/Continuous/Transfer Fcn', [modelName, '/Navigation Dynamics']);
set_param([modelName, '/Navigation Dynamics'], 'Numerator', '[1]', 'Denominator', '[5
1]');
add_block('simulink/Continuous/PID Controller', [modelName, '/Navigation PID
Controller']);
set_param([modelName, '/Navigation PID Controller'], 'P', '1', 'I', '0.5', 'D',
'0.1');
% Connect Navigation blocks
add_line(modelName, 'Navigation Dynamics/1', 'Navigation Error Calculation/2');
add_line(modelName, 'Desired Altitude/1', 'Navigation Error Calculation/1');
add_line(modelName, 'Navigation Error Calculation/1', 'Navigation PID Controller/1');
add_line(modelName, 'Navigation PID Controller/1', 'Navigation Dynamics/1');
% Step 8: Set Simulation Parameters
set_param(modelName, 'Solver', 'ode45', 'StartTime', '0', 'StopTime', '100');
% Step 9: Run the Simulation
sim(modelName);
% Display the results (Scope)
open_system([modelName, '/Altitude Scope']);
open_system([modelName, '/Altitude Error Scope']);
open_system([modelName, '/Navigation Dynamics']);
```

**OUTPUT :**

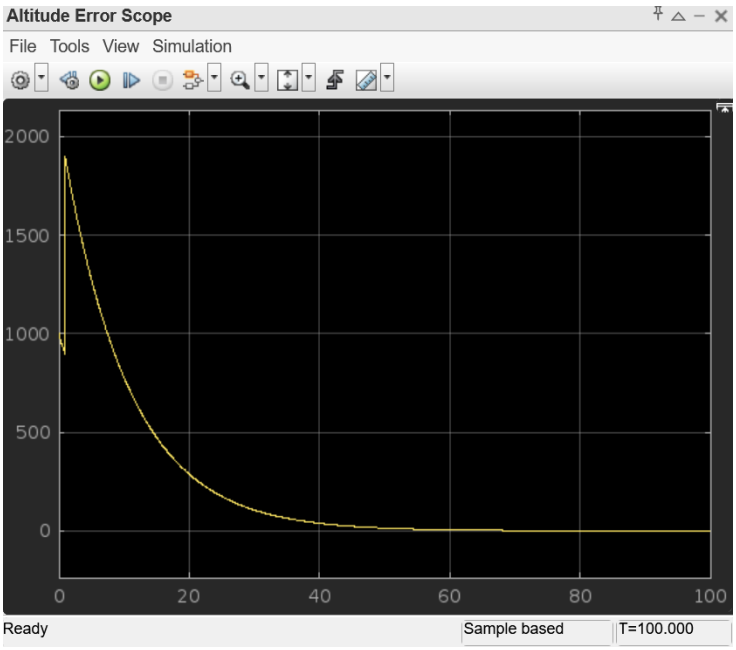


Figure 4.2.1 : Altitude error scope

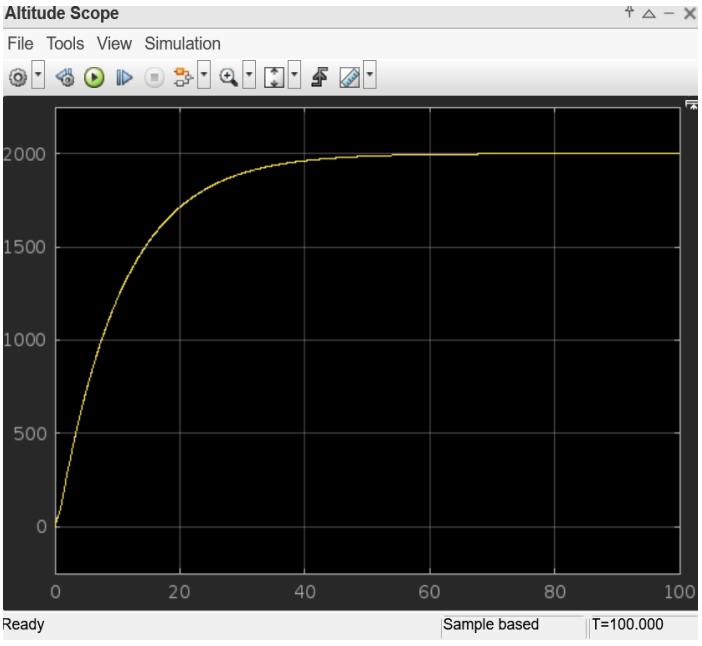


Figure 4.2.2: Altitude scope

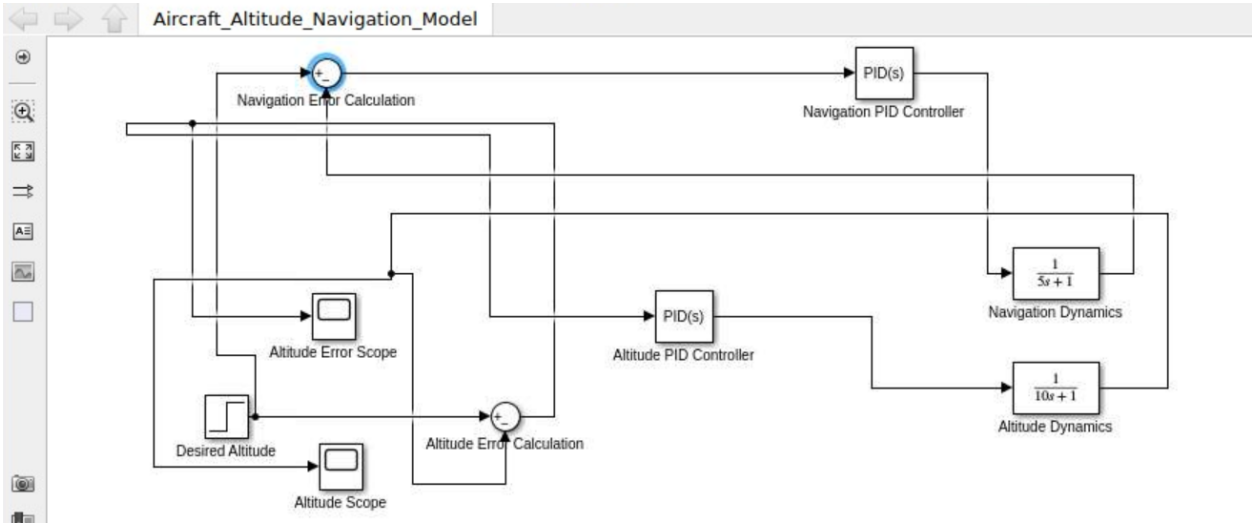


Figure 5.2.3 : Simulation of altitude and navigation

**EXPLANATION OF CODE:**

This MATLAB code simulates the altitude regulation and navigation of an aircraft using Simulink. First, it initializes the system by clearing previous data and defining a new model. The model begins with the addition of an **Altitude Dynamics** block, which represents the aircraft's altitude behavior using a transfer function. A **Step Input** block defines the desired altitude, which changes over time. The difference between the desired and actual altitudes is calculated using a **Sum block**, creating an **altitude error**. This error is fed into a **PID Controller** to adjust the aircraft's altitude. The system's performance is visualized using **Scope blocks**, which show the actual altitude and error. The next step involves adding a **Navigation Dynamics** block, which models the aircraft's horizontal navigation. A **Proportional control** system adjusts the aircraft's course based on navigation errors. The blocks are connected in a signal flow, ensuring that the PID controllers for both altitude and navigation adjust the aircraft's movements to meet the desired conditions. The simulation parameters are set, and the system is run over a period of 100 seconds. Finally, the results are displayed in the Scope blocks, allowing the user to analyze the altitude regulation and navigation performance.

# Ch 4.3 : Flight Path

**CODE:**

```
% MATLAB Code to Simulate Flight Path System for Aircraft
clc
clear all
close all
% Define the model name
modelName = 'FlightPath_Model';
% Check if the model already exists and close it if necessary
if bdIsLoaded(modelName)
close_system(modelName, 0); % Close the model without saving
delete_system(modelName); % Delete the model from memory
end
% Create a new model
new_system(modelName); % Create new model
open_system(modelName); % Open the model
% Step 1: Aircraft Dynamics (for altitude control and flight path)
add_block('simulink/Continuous/Transfer Fcn', [modelName, '/Altitude Dynamics']);
set_param([modelName, '/Altitude Dynamics'], 'Numerator', '[1]', 'Denominator', '[1 1
0]'); % Simplified altitude dynamics
% Step 2: Add Throttle Input (Control for Engine Power)
add_block('simulink/Sources/Step', [modelName, '/Throttle Input']);
set_param([modelName, '/Throttle Input'], 'Time', '1', 'Before', '0', 'After', '10');
% Step input for throttle control
% Step 3: Add PID Controller for Altitude Control
add_block('simulink/Continuous/PID Controller', [modelName, '/Altitude PID
Controller']);
set_param([modelName, '/Altitude PID Controller'], 'P', '1', 'I', '0', 'D', '0');
% Step 4: Add Scope to visualize results (Altitude and Flight Path)
add_block('simulink/Sinks/Scope', [modelName, '/Altitude Scope']);
% Step 5: Connect Blocks for Altitude Control
% Connect Throttle Input to PID Controller (Throttle as input to PID for altitude
control)
add_line(modelName, 'Throttle Input/1', 'Altitude PID Controller/1');
% Connect PID Controller output to Altitude Dynamics input (Altitude command)
add_line(modelName, 'Altitude PID Controller/1', 'Altitude Dynamics/1');
% Connect Altitude Dynamics output to Altitude Scope
add_line(modelName, 'Altitude Dynamics/1', 'Altitude Scope/1');
% Step 6: Set Simulation Parameters
set_param(modelName, 'Solver', 'ode45', 'StartTime', '0', 'StopTime', '50'); % Set
simulation solver and time range
% Step 7: Run the Simulation
sim(modelName);
% Display the results (Scopes)
open_system([modelName, '/Altitude Scope']);
```

**OUTPUT :**

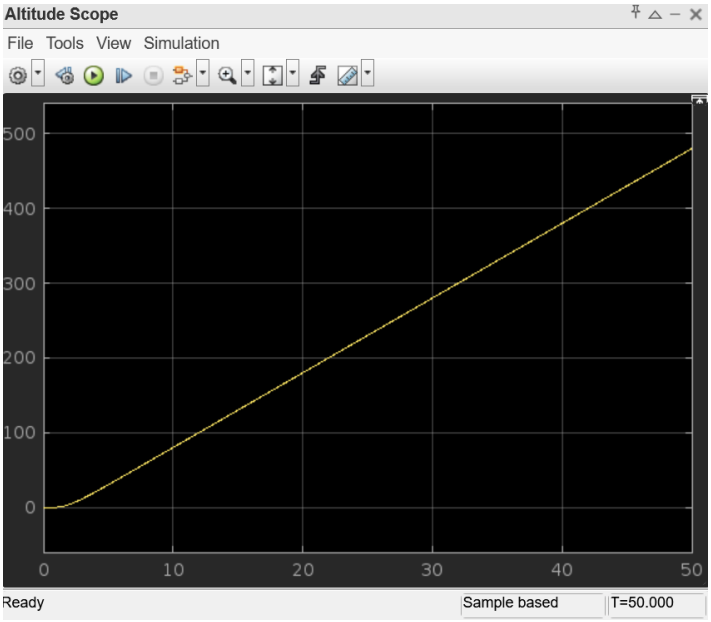


Figure 4.3.1 : Altitude scope for simulated flight path

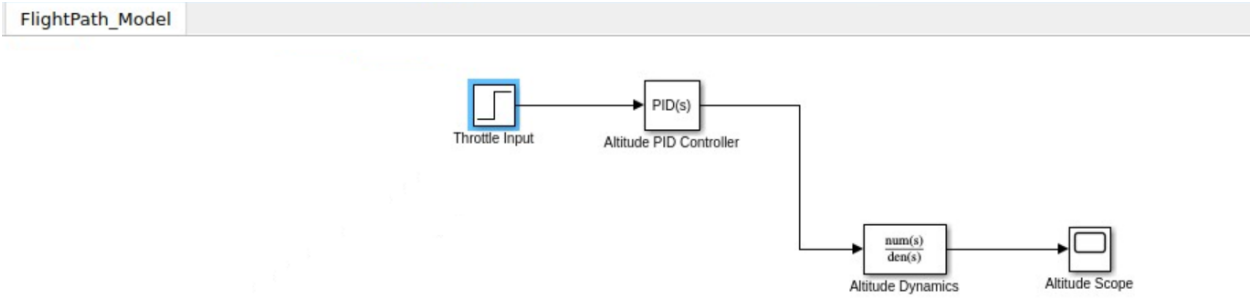


Figure 4.3.2 : Flight path model

**EXPLANATION OF CODE :**

This MATLAB code simulates the **flight path system** of an aircraft, focusing on altitude control. It starts by initializing the system, clearing previous data, and defining a new Simulink model called **FlightPath_Model**. The model begins by adding an **Altitude Dynamics** block, which represents a simplified model of the aircraft's altitude behavior using a transfer function. A **Throttle Input** block is then introduced, simulating control over the engine's power to influence the aircraft's altitude. A **PID Controller** is added next to regulate the altitude by adjusting the throttle input based on the altitude error (difference between desired and actual altitude). The system also includes a **Scope block** to visualize the output, showing both the altitude and flight path behavior during the simulation. The blocks are interconnected, with the throttle input feeding into the PID controller, which adjusts the altitude dynamics. The simulation parameters are set using the **ode45 solver**, and the simulation runs over a period of 50 seconds. After the simulation, the results are displayed in the **Altitude Scope**, allowing for analysis of how the system responds to the throttle input in controlling the aircraft's altitude and flight path.

# Ch 5 :CONCLUSION

This project successfully simulates key aspects of an aircraft's avionics system, specifically focusing on **altitude regulation**, **navigation**, and **flight path control**. Using MATLAB and Simulink, three distinct systems were modeled and integrated to demonstrate how these components work together to ensure stable aircraft operation. The **altitude regulation** system utilized a PID controller to adjust the aircraft's altitude based on the difference between the desired and actual values, with throttle control as the primary input. Similarly, the **navigation system** incorporated a proportional control mechanism to correct the aircraft's horizontal course by minimizing the navigation error. The **flight path system** focused on maintaining altitude and path control through a throttle input managed by a PID controller. Each system was linked and simulated in a unified model, with results visualized using **Scope blocks** to monitor the aircraft’s performance over time. The use of **Transfer Function** blocks for dynamics modeling and **PID controllers** for feedback control ensured that the aircraft could respond dynamically to inputs while maintaining stability. The simulation provided valuable insights into how avionics systems interact in real-time and highlighted the effectiveness of control systems in maintaining optimal flight conditions. Overall, the project demonstrated the critical role of simulation in understanding and designing robust avionics systems for aircraft operations.

# Ch 6 :REFERENCES


1. AVD Avionics System design manual
2. Flight dynamics by Robert F.Stengel
3. Introduction to Avionics system by R.P.G Collinson
4. MATLAB online manual
5. Wikipedia
6. Britannica